

Lab 8

Part A - Mutable default arguments

1. Create a **BadTeam** class with **name** and a default parameter **members=[]**. Add an **add_member()** method.
2. Create two **BadTeam** objects without providing a members list. Add a member to only one team and print both lists. Explain in a comment what happened.
3. Create a corrected **Team** class using **None** as the default value and create a new list inside **__init__**.
4. Repeat the test with two **Team** objects and show that each object now has its own list.

Part B - Dictionary or class?

1. Represent a movie using a dictionary with **title**, **director** and **rating**.
2. Represent the same information using a **Movie** class.
3. Add a method to **Movie** that returns whether the movie is highly rated. Choose a sensible rating threshold.
4. In comments, briefly explain one situation where you would choose a dictionary and one where you would choose a class.

Part C - Inheritance fundamentals

1. Create a base class **Account** with **owner** and **balance**.
2. Create **SavingsAccount(Account)** with an additional **interest_rate** attribute.
3. Use **super()** so **SavingsAccount** reuses the initialization from **Account**.
4. Create at least two objects and print their attributes.
5. Write the "is-a" statement that explains why this inheritance relationship makes sense.

Part D - Inherited and subclass-specific behaviour

1. Create a base class **Employee** with **name** and a method **get_information()**.
2. Create **Developer(Employee)** and add a method that only **Developer** has.
3. Create another **Employee** subclass of your choice and give it its own subclass-specific method.
4. Demonstrate that both subclasses can use inherited behaviour from **Employee**.
5. Demonstrate that an **Employee** object cannot automatically use a method that only exists in one of its subclasses.

Part E - super() and shared initialization

1. Create a base class **Device** with **brand** and **year**.
2. Add useful shared initialization logic inside **Device**, for example validation that **year** cannot be negative and an attribute such as **is_active=True**.
3. Create **Laptop(Device)** with one additional attribute such as **ram_gb**. Use **super()**.
4. Create another **Device** subclass with its own additional attribute and use **super()** again.

5. Demonstrate that both subclasses receive the shared initialization logic from Device without duplicating it.

Part F - Method overriding

1. Create a base class **Notification** with a method **send()** that returns a general message.
2. Create **EmailNotification(Notification)** and **SMSNotification(Notification)**.
3. Override **send()** in both subclasses so each returns a different message.
4. Create one object from each class and call **send()** on all of them.
5. Explain in a comment which method is used when **send()** is called on each object.

Part G - Override and still use the base method

1. Create a base class **Report** with a method **get_summary()** that returns a general report summary.
2. Create **SalesReport(Report)** and override **get_summary()**.
3. Inside the overridden method, call the base implementation using **super()** and add SalesReport-specific information.
4. Create a SalesReport object and print the final result.

Part H - Applied challenge: User accounts

1. Build a small user account system using inheritance.
2. Create a base class **User** with at least **username** and **email**.
3. Add a useful method to User that all user types should inherit.
4. Create **AdminUser(User)** and **PremiumUser(User)**. Give each subclass at least one additional attribute and one subclass-specific method.
5. Use **super()** in both subclasses instead of duplicating User's initialization.
6. Add one method to User and override it differently in AdminUser and PremiumUser.
7. In one overridden method, use **super()** to reuse the base implementation and then extend it.
8. Create several objects and demonstrate inherited methods, subclass-specific methods and overridden methods.
9. Add at least one sensible validation using **ValueError**.
10. In comments, explain why AdminUser and PremiumUser have an "is-a" relationship with User.